LAB EXPERIMENT

NAME : B Hitesh Roy

REG NO : 1923272036

COURSE CODE: (CSA0618)

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHM.

Experiment 1: Linear Search (Iterative)

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 25, 30, 45, 50]
key = 30

result = linear_search(arr, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

Experiment 2: Binary Search (Recursive)

```
def binary_search(arr, low, high, key):
```

```
    if low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            return binary_search(arr, low, mid - 1, key)
        else:
            return binary_search(arr, mid + 1, high, key)

    return -1

arr = [5, 10, 15, 20, 25]
key = 20

result = binary_search(arr, 0, len(arr) - 1, key)

if result != -1:
    print("Key found at index", result)
else:
    print("Key not found")
```

Experiment 3: Factorial (Iterative vs Recursive)

```
# Iterative
def factorial_iterative(n):
    fact = 1
```

```
    for i in range(1, n + 1):
        fact *= i
    return fact

# Recursive
def factorial_recursive(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial_recursive(n - 1)

n = 5

print("Iterative:", factorial_iterative(n))
print("Recursive:", factorial_recursive(n))
```

Experiment 4: Fibonacci Series

```
# Iterative
def fibonacci_iterative(n):
    series = []
    a, b = 0, 1
    for _ in range(n):
        series.append(a)
        a, b = b, a + b
    return series

# Recursive
```

```
def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

n = 6

print("Iterative:", fibonacci_iterative(n))

print("Recursive:")
for i in range(n):
    print(fibonacci_recursive(i), end=" ")
```

Experiment 5: Matrix Multiplication

```
A = [[1, 2],
     [3, 4]]

B = [[5, 6],
     [7, 8]]

result = [[0, 0],
          [0, 0]]

for i in range(len(A)):
    for j in range(len(B[0])):
        for k in range(len(B)):
```

```
            result[i][j] += A[i][k] * B[k][j]

print(result)
```

Experiment 6: Merge Sort

```
def merge_sort(arr):
    if len(arr) > 1:

        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
```

```
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

arr = [38, 27, 43, 3, 9, 82, 10]

merge_sort(arr)
print(arr)
```

Experiment 7: Quick Sort

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr)//2]

    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
```

```
    return left + middle + right

arr = [10, 7, 8, 9, 1, 5]

print(quick_sort(arr))
```

Experiment 8: Tower of Hanoi

```
def tower_of_hanoi(n, source, auxiliary, destination):
    if n == 1:
        print("Move disk 1 from", source, "to", destination)
        return

    tower_of_hanoi(n - 1, source, destination, auxiliary)

    print("Move disk", n, "from", source, "to", destination)

    tower_of_hanoi(n - 1, auxiliary, source, destination)

n = 3

tower_of_hanoi(n, 'A', 'B', 'C')
```

Experiment 9: GCD (Euclidean Algorithm)

```
def gcd(a, b):
    if b == 0:
        return a
```

```
    return gcd(b, a % b)

a = 48
b = 18

print("GCD =", gcd(a, b))
```

Experiment 10: Insertion Sort

```
def insertion_sort(arr):

    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

arr = [12, 11, 13, 5, 6]

insertion_sort(arr)

print(arr)
```

Experiment 11: Heap Sort

```
def heapify(arr, n, i):
    largest = i

    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
```

```
arr = [4, 10, 3, 5, 1]

heap_sort(arr)

print(arr)
```

Experiment 12: Counting Inversions

```
def merge_count(arr):
    if len(arr) <= 1:
        return arr, 0

    mid = len(arr) // 2

    left, inv_left = merge_count(arr[:mid])
    right, inv_right = merge_count(arr[mid:])

    merged = []
    i = j = 0
    inv = inv_left + inv_right

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
```

```
            merged.append(right[j])
            inv += len(left) - i
            j += 1

    merged.extend(left[i:])
    merged.extend(right[j:])

    return merged, inv

arr = [2, 4, 1, 3, 5]

sorted_arr, inversions = merge_count(arr)

print("Inversions =", inversions)
```

Experiment 13: Maximum Subarray Sum (Kadane's Algorithm)

```
def kadane(arr):
    max_current = max_global = arr[0]

    for i in arr[1:]:
        max_current = max(i, max_current + i)

        if max_current > max_global:
            max_global = max_current

    return max_global
```

```
arr = [-2, -3, 4, -1, -2, 1, 5, -3]

print("Maximum Sum =", kadane(arr))
```

Experiment 14: Exponentiation (Fast Power)

```
# Iterative
def power_iterative(x, n):
    result = 1

    for _ in range(n):
        result *= x

    return result

# Recursive
def power_recursive(x, n):
    if n == 0:
        return 1

    half = power_recursive(x, n // 2)

    if n % 2 == 0:
        return half * half
    else:
        return x * half * half
```

```
x = 2
n = 10

print("Iterative:", power_iterative(x, n))
print("Recursive:", power_recursive(x, n))
```

Experiment 15: Closest Pair of Points (Brute Force)

```
import math

def closest_pair(points):
    min_dist = float('inf')
    pair = None

    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            dist = math.sqrt((points[i][0] - points[j][0])**2 +
                             (points[i][1] - points[j][1])**2)

            if dist < min_dist:
                min_dist = dist
                pair = (points[i], points[j])

    return pair, min_dist

points = [(1, 2), (4, 5), (7, 8), (3, 1)]
```

```
pair, distance = closest_pair(points)


print("Closest Pair:", pair)
print("Distance:", distance)
```